Assignment I

Problem Bank 20

Assignment Description:

The assignment aims to provide deeper understanding of cache by analysing its behaviour using cache implementation of CPU- OS Simulator. The assignment has three parts.

- Part I deals with Cache Memory Management with Direct Mapping
- Part II deals with Cache Memory Management with Associative Mapping
- Part III deals with Cache Memory Management with Set Associative Mapping

- **Submission:** You will have to submit this documentation file and the name of the file should be GROUP-NUMBER.pdf. For Example, if your group number is 1, then the file name should be GROUP-1.pdf.

Submit the assignment by **15th June 2025 through Taxila only**. File submitted by any means outside TAXILA will not be accepted and marked. In case of any issues, please drop an email to the course lead LF Vaibhav Jain (email: vaibhav.jain@wilp.bits-pilani.ac.in).

Caution!!!

Assignments are designed for individual groups which may look similar, and you may not notice minor changes in the assignments. Hence, refrain from copying or sharing documents with others. Any evidence of such practice will attract severe penalty. Remember that any kind of group changes after the announcement of the assignment is not allowed.

Evaluation:

- The assignment carries 10 marks
- Grading will depend on
    - Contribution of each student in the implementation of the assignment
    - **Plagiarism or copying will result in -10 marks**

**************************FILL IN THE DETAILS GIVEN BELOW****************

Assignment Set Number:1

Group Name: Group120

Contribution Table:

Contribution (This table should contain the list of all the students in the group. Clearly mention each student's contribution towards the assignment. Mention "No Contribution" in cases applicable.)

Sl. No.	Name (as appears in Taxila)	ID NO	Contribution
1.	KANHAIYA SHARMA	2024DC04237	1/3
2.	AKASH MALIK	2024DC04030	1/3
3.	RAJESHWARI M	2024DC04277	1/3

Resource for Part I, II and III:

- Use following link to login to "eLearn" portal.
  - https://elearn.bits-pilani.ac.in
- Under tab 'My Courses', select Labware and then go to the course page "**Labware_Computer Organization and Software Systems**". The direct link to access the course is https://taxila-aws.bits-pilani.ac.in/course/view.php?id=12598
- Click on "CSIS-Connect to Virtual Lab(OSHA)" to login to lab portal using elearn credentials.
- Use Cloud virtual lab access manual to use the lab platform and access the Cloud virtual machine.
- In cloud lab machine, click on "Lab Resources" -> Click on "Computer Organization and software systems" course folder.
  - Use resources within "LabCapsule3: Cache Memory"

Code to be used:

The following code written in STL Language:

```
program PB20_Group210
        var a array(75) byte
                writeln("Array Elements: ")

                for n = 0 to 14
                        a(n) = n+10
                        writeln(a(n))
                next
        key = 60
        writeln("Key to be searched: ",key)

        found = 0
        for n = 0 to 14
        temp = a(n)
        if temp = key then
        found = 1
        writeln("Key Found",temp)
        break
        end if
        next
        if found <> 1 then
        writeln("Key Not Found")
        end if
        end
```

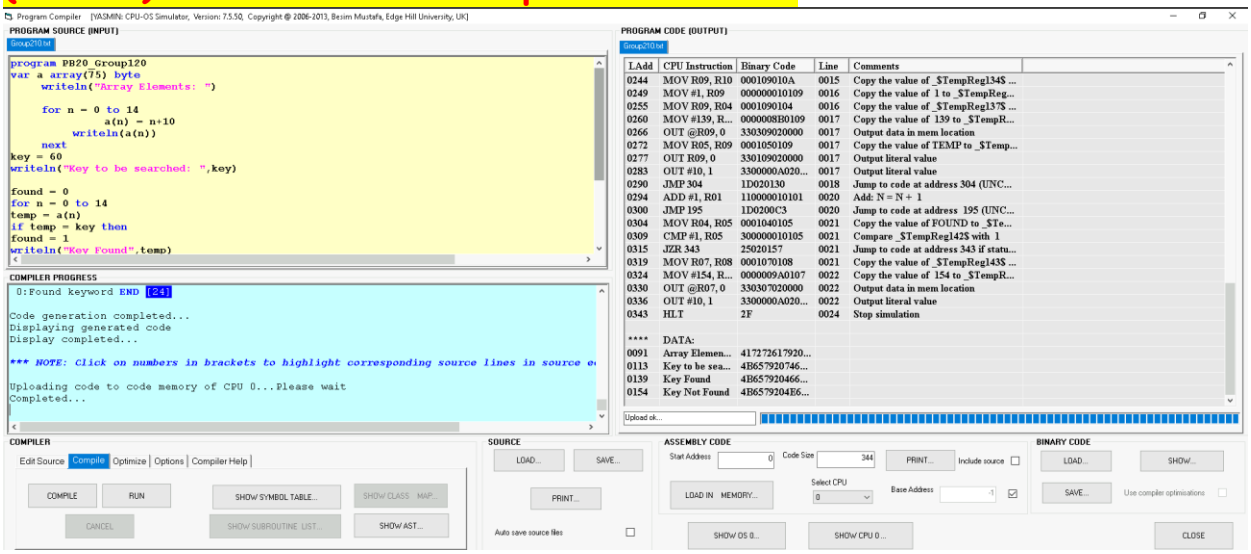
General procedure to convert the given STL program into ALP:

- Open CPU OS Simulator. Go to **advanced tab** and press **compiler** button
- Copy the above program in **Program Source** window
- Replace <PB20_GroupName> by "**PB20_ followed by your group number**". For example, if group number is Group99 then PB20_Group99
- Open **Compile** tab and press **compile** button
- In **Assembly Code,** enter **start address** and press **Load in Memory** button
- Now the assembly language program is available in CPU simulator.
- Set speed of execution to **FAST.**
- Open I/O console
- To run the program press **RUN** button.

General Procedure to use Cache set up in CPU-OS simulator

- After compiling and loading the assembly language code in CPU simulator, press "Cache-Pipeline" tab and select cache type as "Data". Press "SHOW CACHE" button.
- In the newly opened cache window, choose appropriate cache Type, cache size, set blocks, replacement algorithm according to the instructions as given in each part.
- The write policy is by-default used as write back policy.

At first, present the snapshot of your PROGRAM SOURCE (INPUT) Window of the compiler simulator.

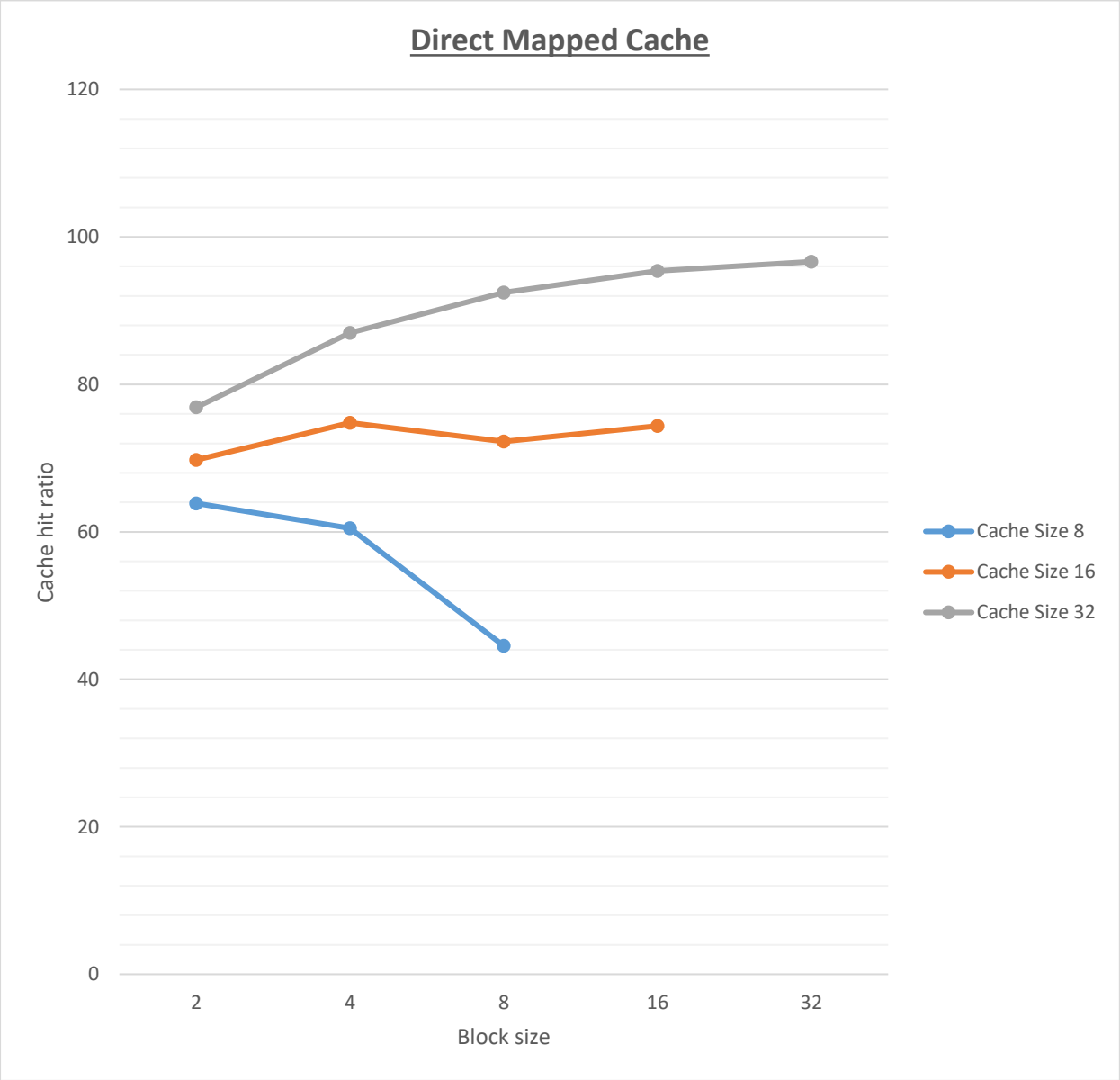


Part I: Direct Mapped Cache

a) Execute the above program by setting block size to 2, 4, 8, 16 and 32 for cache size = 8, 16, and 32. Record the observation in the following table.

Block Size	Cache size	# Hits	# Misses	% Miss Ratio	%Hit Ratio
2	8	152	86	36.13	63.87
4		144	94	39.50	60.50
8		106	132	55.46	44.54
2	16	166	72	30.25	69.75
4		178	60	25.21	74.79
8		172	66	27.73	72.27
16		177	61	25.63	74.37
2	32	183	55	23.11	76.89
4		207	31	13.03	86.97
8		220	18	7.56	92.44
16		227	11	4.62	95.38
32		230	8	3.36	96.64

a) Plot a single graph of Cache hit ratio Vs Block size with respect to cache size = 8, 16 and 32. Comment on the graph that is obtained.



Observation From the Graph of Direct Mapped Cache.

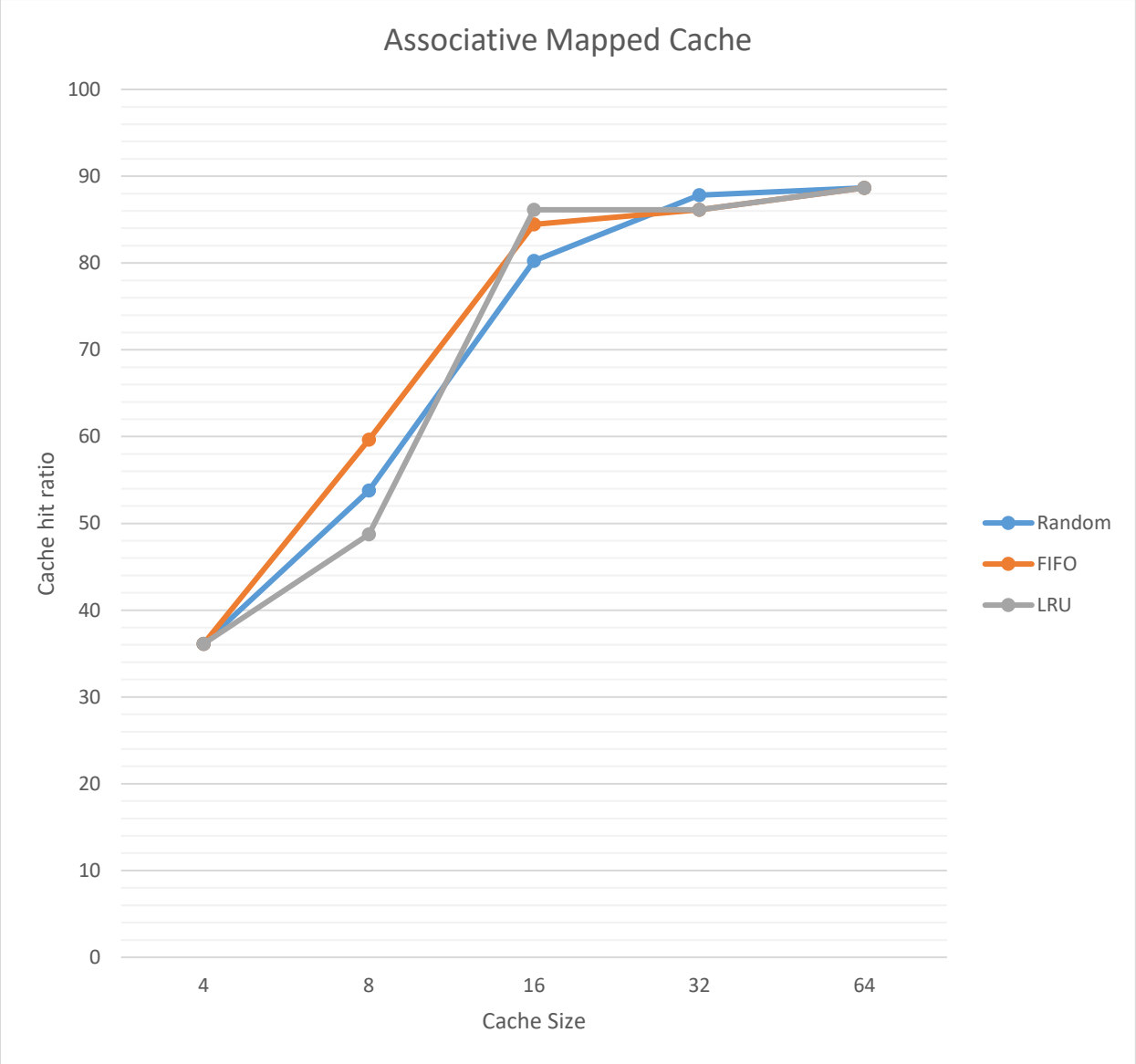
- Larger cache sizes result in higher hit ratios across all block sizes.
- Optimal Block Size Varies by Cache Size. For smaller caches (e.g., size 8), increasing block size reduces the hit ratio. In contrast, for larger caches (e.g., size 32), larger block sizes increase hit ratio.
- The increase in hit ratio from block size 16 to 32 is very small (95.38% to 96.64%), indicating diminishing improvement despite increased block size.

Part II: Associative Mapped Cache

a) Use the given program, fill up the following table for three different replacement algorithms and state which replacement algorithm is better and why?

Replacement Algorithm: Random				
Block Size	Cache size	Miss	Hit	Hit ratio
4	4	152	86	36.13
4	8	110	128	53.78
4	16	47	191	80.25
4	32	29	209	87.82
4	64	27	211	88.66
Replacement Algorithm: FIFO				
Block Size	Cache size	Miss	Hit	Hit ratio
4	4	152	86	36.13
4	8	96	142	59.66
4	16	37	201	84.45
4	32	33	205	86.13
4	64	27	211	88.66
Replacement Algorithm: LRU				
Block Size	Cache size	Miss	Hit	Hit ratio
4	4	152	86	36.13
4	8	122	116	48.74
4	16	33	205	86.13
4	32	33	205	86.13
4	64	27	211	88.66

b) Plot a single graph of Cache Hit Ratio Vs Cache size with respect to different replacement algorithms. Comment on the graph that is obtained.



Observation From the Graph of Associative Mapped Cache.

- As the cache size increases from 4 to 64, the hit ratios for all three policies (Random, FIFO, LRU) steadily increase.
- At cache size 4 (smallest) and cache size 64 (largest), the hit ratios for Random, FIFO, and LRU are equal. This convergence suggests that the impact of the cache replacement policy diminishes when the cache size is either very limited or very ample.

Part III: Set Associative Mapped Cache

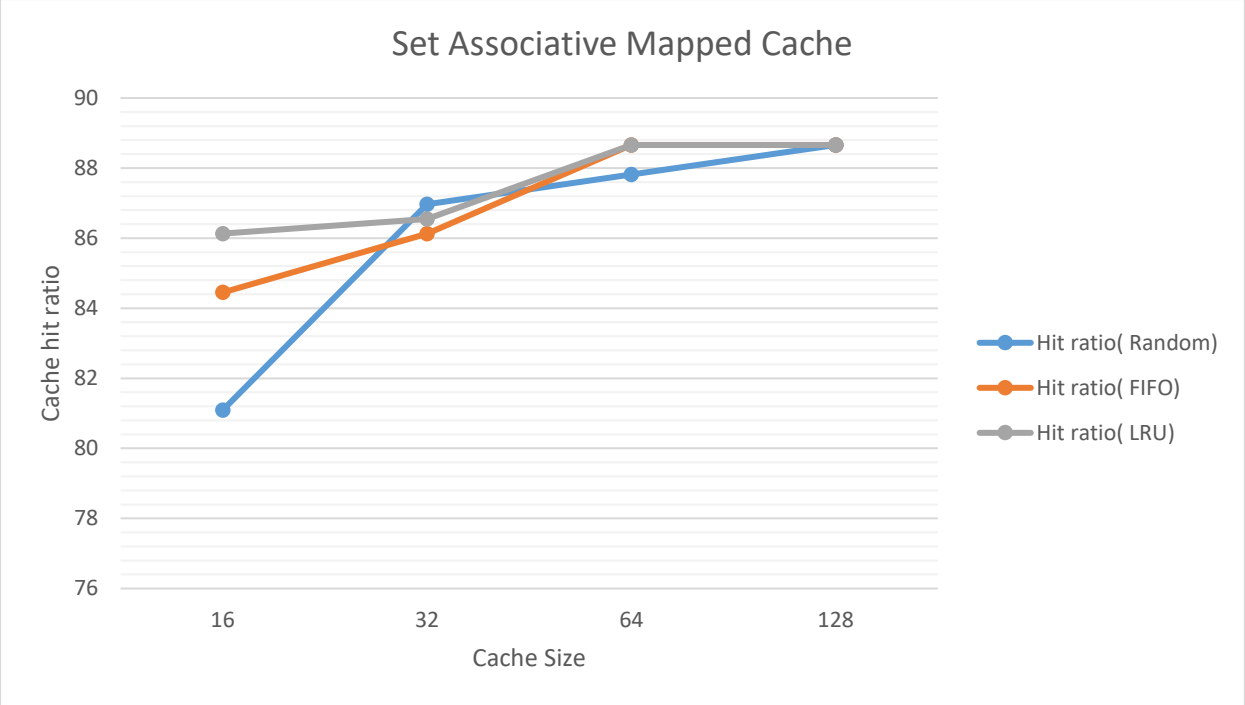
Execute the above program by setting the following Parameters:

- Number of sets (Set Blocks): 4 way
- Cache Type: Set Associative
- Replacement: LRU/FIFO/Random

a) Fill up the following table for three different replacement algorithms and state which replacement algorithm is better and why?

Replacement Algorithm: Random				
Block Size	Cache size	Miss	Hit	Hit ratio
4	16	45	193	81.09
4	32	31	207	86.97
4	64	29	209	87.82
4	128	27	211	88.66
	Replacement Algorithm: FIFO			
Block Size	Cache size	Miss	Hit	Hit ratio
4	16	37	201	84.45
4	32	33	205	86.13
4	64	27	211	88.66
4	128	27	211	88.66
	Replacement Algorithm: LRU			
Block Size	Cache size	Miss	Hit	Hit ratio
4	16	33	205	86.13
4	32	32	206	86.55
4	64	27	211	88.66
4	128	27	211	88.66

b) Plot a single graph of Cache Hit Ratio Vs Cache size with respect to different replacement algorithms. Comment on the graph that is obtained.



Observation From the Graph of Set Associative Mapped Cache.

- The hit ratio for all policies improves as the cache size grows from 16 to 128, but the increase slows down significantly near size 64, indicating diminishing returns on hit ratio gains with very large caches.

c) Fill in the following table and analyse the behaviour of Set Associate Cache. Which one is better and why?

Replacement Algorithm: LRU				
Block Size, Cache size	Set Blocks	Miss	Hit	Hit ratio
4, 64	2 – Way	27	211	88.66
4, 64	4 – Way	27	211	88.66
4, 64	8 – Way	28	210	88.24

Observation From the table of Replacement Algorithm: LRU:

2-Way Set Associative Cache using the LRU replacement algorithm is the most efficient choice in this scenario. It delivers the **highest hit ratio (88.66%) with fewer misses (27)**, while avoiding the increased **hardware complexity and overhead** associated with higher associativity levels like 4-Way and 8-Way. Since higher associativity does not provide a performance gain here, 2-Way strikes the best balance between **performance and simplicity**.